

Aplicación móvil Café del Centro: diseño UX/UI en Figma y desarrollo con React Native

Cristian Arturo Hernández Flores (HF252375)

Universidad Don Bosco

Diseño y Programación de Software Multiplataforma

Ing. Alexander Alberto Sigüenza Campos

18 de octubre de 2026

Índice

Introducción 4

Objetivos 4

Objetivo general 4

Objetivos específicos 5

React Native en el desarrollo de la aplicación 5

¿Qué es React Native? 5

Expo y Expo Router 5

De la web al móvil 6

Manejo del estado con Context 7

Arquitectura de la aplicación 7

Figma en el proceso de diseño 8

Figma como herramienta de diseño 8

Sistema visual definido en la Etapa 1 8

Del prototipo a la aplicación: decisiones de diseño 9

Una aplicación enfocada en el cliente 9

Navegación por pestañas inferiores 9

Del menú estático al catálogo interactivo 11

Pantallas nuevas con el mismo sistema visual 12

Un sistema de diseño centralizado 12

Implementación del avance 12

Autenticación 13

Menú y selección de tamaño 13

Carrito y pago 14

Confirmación y cuenta 14

Validaciones de formularios 14

Componentes reutilizables 15

Limitaciones y trabajo futuro 16

Conclusiones 17

Referencias 18

Introducción

Café del Centro es una cafetería de especialidad ubicada en el Centro Histórico de San Salvador, a pocos pasos de la Plaza Barrios. Desde 2023 trabaja con café de fincas salvadoreñas y fue reconocida en el puesto número 59 entre las mejores cafeterías de Norteamérica, Centroamérica y el Caribe. Para este proyecto se desarrolla un sistema de pedidos que permite a los clientes elegir sus bebidas, seleccionar el tamaño del vaso, pagar su pedido y acumular puntos de fidelidad.

En la Etapa 1 se analizó el negocio y se diseñaron los mockups en Figma. En la Etapa 2 se construyó la versión web con React y Next.js, que incluye un panel de administración para gestionar el catálogo y los pedidos. Este documento corresponde al primer avance de la versión móvil, desarrollada con React Native y Expo, y pensada para el cliente que hace su pedido desde el teléfono.

A lo largo del documento se explica cómo se aplicaron React Native y Figma en el proyecto. Primero se describen las herramientas utilizadas y la arquitectura de la aplicación. Después se presenta el sistema visual definido en Figma y se justifican las decisiones de diseño que se tomaron al llevarlo al teléfono. Por último, se resume la implementación del avance, las validaciones de formularios, las limitaciones encontradas y el trabajo pendiente para las siguientes entregas.

Objetivos

Objetivo general

Desarrollar la primera versión de la aplicación móvil de Café del Centro con React Native, aplicando el sistema visual definido en Figma durante la Etapa 1 y adaptándolo a las necesidades del cliente en un dispositivo móvil.

Objetivos específicos

- Implementar el registro y el inicio de sesión de clientes con validación de formularios.
- Construir la navegación principal de la aplicación y el flujo completo de pedido, desde el menú hasta la confirmación.
- Mantener la coherencia visual con los mockups de Figma por medio de un sistema de diseño centralizado y componentes reutilizables.
- Justificar las decisiones de diseño tomadas al adaptar los mockups al contexto móvil.

React Native en el desarrollo de la aplicación

¿Qué es React Native?

React Native es un framework de código abierto creado por Meta que permite desarrollar aplicaciones móviles con JavaScript o TypeScript, usando los mismos principios de React (Meta Platforms, s. f.). A diferencia de una página web, la aplicación no muestra HTML dentro de un navegador: cada componente se convierte en un elemento nativo del sistema operativo. De esta forma, con una sola base de código, la aplicación funciona en Android y en iOS.

Para el proyecto, esta característica fue determinante. Como la versión web ya se había desarrollado con React, los conceptos de componentes, propiedades y estado se aprovecharon de forma directa. Además, escribir una sola aplicación para ambas plataformas es la idea central del desarrollo multiplataforma que se busca en la materia.

Expo y Expo Router

Para crear el proyecto se utilizó Expo, un conjunto de herramientas que simplifica el desarrollo con React Native. Durante el desarrollo no fue necesario compilar código nativo:

la aplicación se probó en el emulador de Android por medio de Expo Go, y cada cambio en el código se reflejaba en pocos segundos gracias a la recarga rápida.

La navegación se construyó con Expo Router, que define las pantallas a partir de los archivos del proyecto (Expo, s. f.). Cada archivo dentro de la carpeta src/app es una pantalla, y las carpetas entre paréntesis, como (tabs), agrupan pantallas sin cambiar la ruta. Se eligió esta herramienta porque funciona de la misma manera que el App Router de Next.js que se usó en la versión web, por lo que la organización de la aplicación móvil resultó familiar desde el inicio.

De la web al móvil

Aunque la lógica de React es la misma, los elementos visuales cambian. En React Native no existen etiquetas como div o p, ni clases de CSS; en su lugar se usan componentes propios del framework y los estilos se escriben como objetos de JavaScript. La Tabla 1 resume las equivalencias que se aplicaron al pasar del proyecto web al móvil.

Tabla 1

Equivalencias entre la versión web y la versión móvil

Versión web (Next.js)	Versión móvil (React Native)	Uso en el proyecto
div	View	Contenedores y tarjetas
p, h1, span	Text	Todo el texto de la aplicación
button	Pressable	Botones, filas del menú y opciones de pago
input	TextInput	Campos de inicio de sesión, registro y tarjeta
img	Image	Fotografías de los productos
Clases de Tailwind CSS	StyleSheet	Estilos de cada componente
Desplazamiento del navegador	ScrollView	Pantallas con contenido largo

Manejo del estado con Context

La aplicación necesita compartir información entre varias pantallas. Por ejemplo, el carrito se llena desde el menú, se revisa en su propia pestaña y se vacía al confirmar el pago. Para resolverlo, se usó la API de Context de React, que permite que varios componentes accedan a los mismos datos sin pasarlos de uno en uno (React, s. f.).

Se crearon dos contextos. El contexto de sesión guarda al usuario que inició sesión y sus puntos, y se almacena en el dispositivo con AsyncStorage (React Native Async Storage, s. f.), de modo que la sesión se mantiene aunque el cliente cierre la aplicación. El contexto del carrito guarda los productos, el tamaño elegido, la cantidad y el total del pedido.

Arquitectura de la aplicación

El proyecto se organizó en capas, igual que la versión web. Cada carpeta tiene una responsabilidad definida:

- **app:** las pantallas y la navegación.
- **components:** las piezas visuales reutilizables, como botones, tarjetas y campos de texto.
- **context:** el estado compartido de sesión y carrito.
- **services:** las peticiones a la API.
- **constants, utils y hooks:** los colores, las funciones de formato, las validaciones y la lógica compartida.

La ventaja principal de esta organización es que las pantallas no saben cómo se obtienen los datos. Si una pantalla necesita el menú, llama a una función de la capa de servicios, y esa función es la que se comunica con la API. Esto permitió reutilizar casi sin cambios la capa de servicios de la versión web, y también facilitará la siguiente entrega, ya que para conectar la aplicación a Firebase bastará con modificar los servicios sin tocar el diseño de las pantallas.

Figma en el proceso de diseño

Figma como herramienta de diseño

Figma es una herramienta de diseño de interfaces que funciona en el navegador y permite que varias personas trabajen sobre el mismo archivo al mismo tiempo (Figma, s. f.). En la Etapa 1 se utilizó para diseñar las vistas del sitio de Café del Centro en su versión de escritorio y en su versión móvil: inicio, carta, nosotros, galería y contacto. Trabajar en Figma antes de programar permitió definir el aspecto del proyecto una sola vez y usarlo como referencia tanto en la web como en la aplicación.

Sistema visual definido en la Etapa 1

Más allá de las pantallas, lo más valioso que dejó el diseño en Figma fue un sistema visual coherente. Sus elementos principales son los siguientes:

- **Paleta de colores:** un fondo crema (#FBF6EE) que transmite calidez, un café oscuro (#3B2A1E) para las secciones destacadas y un tono caramelo (#C8873F) para botones, precios y acentos.
- **Tipografía:** una fuente con serifa para los títulos, que da un aire clásico acorde al Centro Histórico, y una fuente sin serifa para el texto, más fácil de leer en pantallas pequeñas.
- **Componentes:** tarjetas blancas con esquinas redondeadas, botones en forma de píldora y una etiqueta pequeña en mayúsculas, de color caramelo, sobre cada título, acompañada de una línea corta debajo.
- **Elementos de identidad:** la tarjeta del reconocimiento “No. 59” y las secciones de la carta, como “Clásicos Calientes” y “Especialidades Frías”.

Del prototipo a la aplicación: decisiones de diseño

Los mockups de la Etapa 1 se diseñaron para un sitio web informativo. Al llevarlos a una aplicación de pedidos, se mantuvo el sistema visual completo, pero fue necesario adaptar la estructura y el contenido. A continuación, se explica cada decisión.

Una aplicación enfocada en el cliente

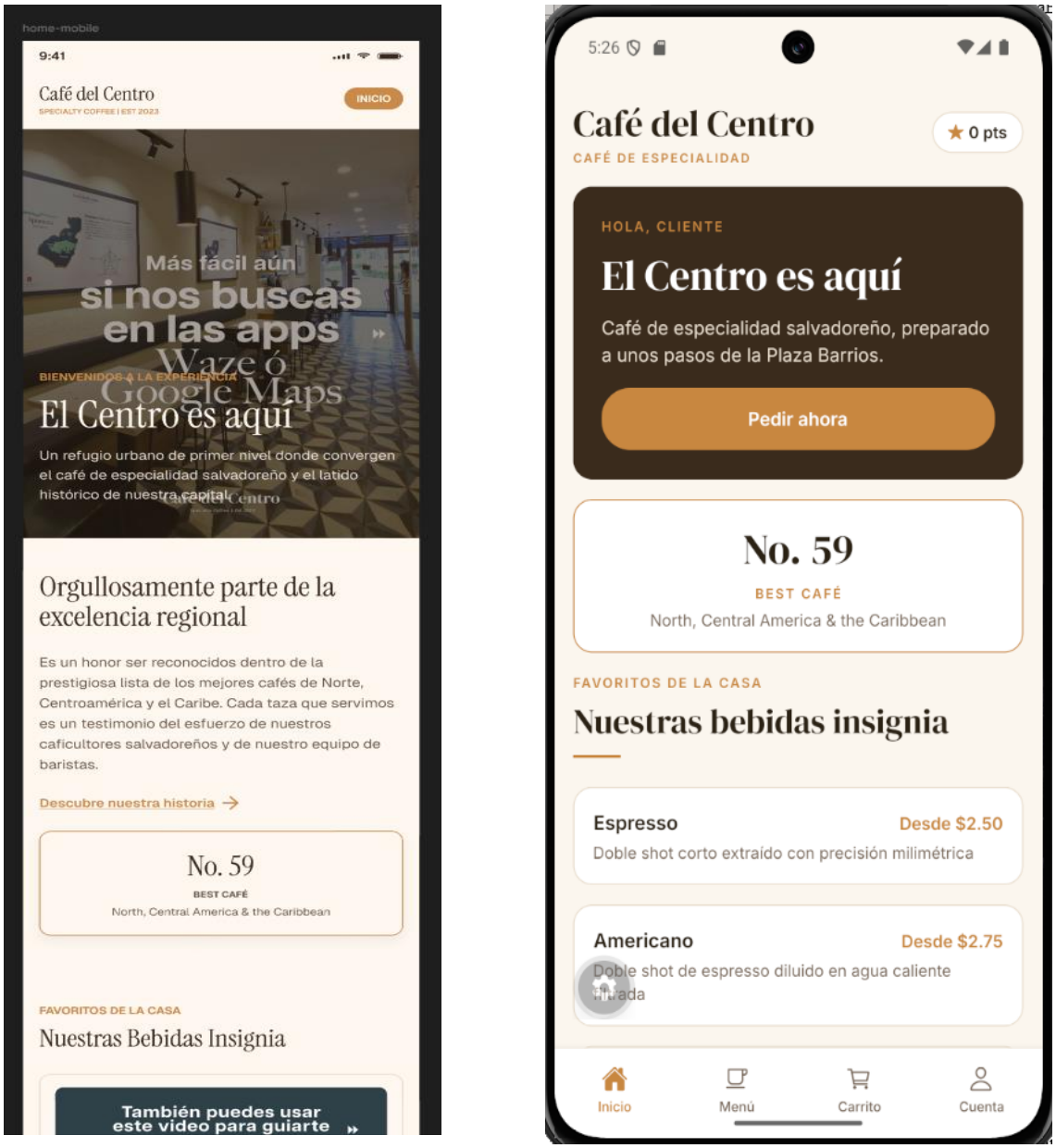
La versión web atiende a dos tipos de usuario: el cliente, que hace pedidos, y el administrador, que gestiona el catálogo y cambia el estado de los pedidos. Se decidió que la aplicación móvil fuera solo para el cliente. El administrador trabaja desde la caja o el mostrador, donde una pantalla grande es más cómoda para revisar pedidos y editar productos. El cliente, en cambio, pide desde su teléfono, muchas veces mientras va de camino a la cafetería. Por esa razón, si una cuenta de administrador intenta entrar a la aplicación, se le indica que el panel de administración está en la versión web.

Navegación por pestañas inferiores

En los mockups, la navegación es un menú en la parte superior con las secciones del sitio. En la aplicación se reemplazó por una barra de pestañas en la parte inferior, con cuatro secciones: Inicio, Menú, Carrito y Cuenta. Las guías de diseño de Android y de iOS recomiendan este patrón para las secciones principales de una aplicación (Apple, s. f.; Google, s. f.). Además, la mayoría de las personas usa el teléfono con una sola mano, y la parte inferior de la pantalla es la zona que se alcanza con el pulgar (Hoober, 2013). La Figura 1 compara la pantalla de inicio del mockup con la de la aplicación.

Figura 1

Comparación de la pantalla de inicio en Figma y en la aplicación



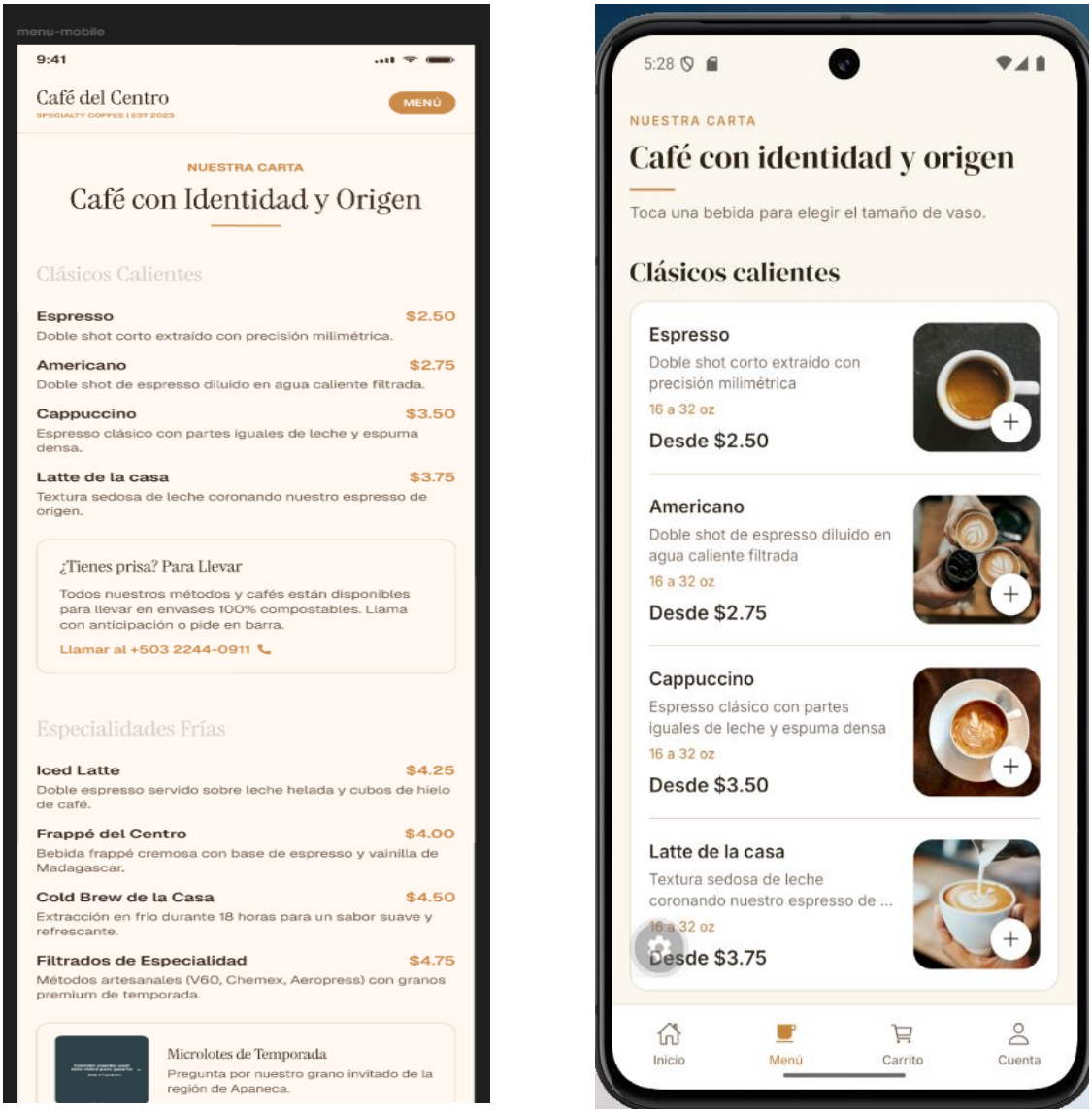
En la pantalla de inicio se conservaron los elementos con más identidad del mockup: el bloque principal en café oscuro con el mensaje “El Centro es aquí”, la tarjeta del “No. 59” y la sección “Nuestras Bebidas Insignia”. También se incluyó el horario de atención, que en el mockup aparecía en el pie de página y en la sección de contacto, porque es la información que el cliente necesita antes de ir a recoger su pedido.

Del menú estático al catálogo interactivo

En el mockup, la carta muestra cada bebida con un solo precio, como un menú impreso. En la aplicación, el cliente necesita elegir el tamaño del vaso (16, 20, 24 o 32 onzas), y cada tamaño tiene un precio diferente. Se mantuvo la organización por secciones, pero cada producto ahora muestra su fotografía y un botón para agregarlo. Al tocarlo, se abre una hoja desde la parte inferior donde el cliente elige el tamaño y la cantidad, y el precio se actualiza en el momento. La Figura 2 muestra este cambio.

Figura 2

Comparación de la carta en Figma y del menú en la aplicación



Se eligió la hoja inferior en lugar de una pantalla nueva para que el cliente no pierda de vista el menú y pueda seguir agregando bebidas con pocos toques. Además, cada vez que agrega un producto, un aviso breve en la parte inferior confirma que se añadió al carrito, sin interrumpir con ventanas emergentes.

Pantallas nuevas con el mismo sistema visual

El sitio de la Etapa 1 no tenía inicio de sesión, carrito, pago ni historial de pedidos, porque era informativo. Para la aplicación, estas pantallas se diseñaron siguiendo las mismas reglas del mockup: fondo crema, títulos con serifa, la etiqueta en caramelo sobre cada título y tarjetas blancas redondeadas. Así, aunque son pantallas que no estaban en Figma, se sienten parte del mismo producto.

Un sistema de diseño centralizado

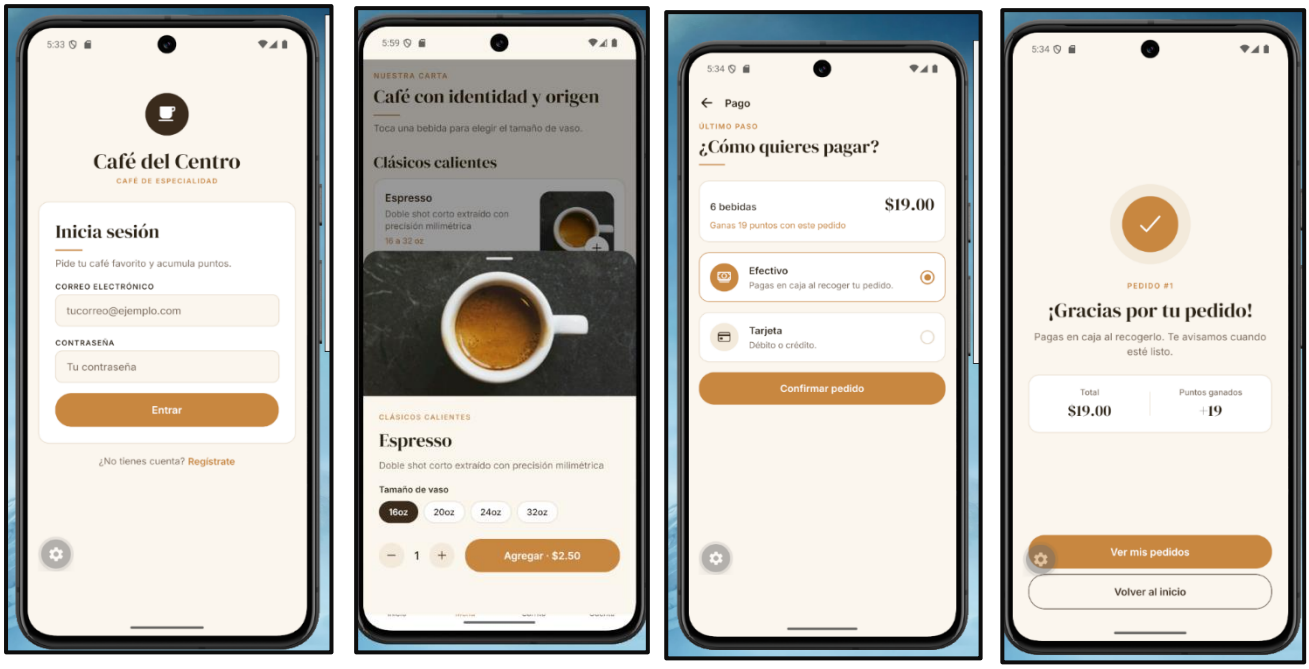
Para que la aplicación se mantuviera fiel a Figma en todas las pantallas, los colores, los espaciados, los radios de las esquinas y las fuentes se guardaron en un solo archivo, theme.ts. Ninguna pantalla define sus propios colores, de modo que si en algún momento cambia el tono caramelo, se modifica en un solo lugar y se actualiza en toda la aplicación. Para la tipografía se usó DM Serif Display en los títulos e Inter en el texto, que siguen el mismo criterio de fuente con serifa y fuente sin serifa definido en los mockups.

Implementación del avance

La Figura 3 resume el flujo principal de la aplicación, desde el inicio de sesión hasta la confirmación del pedido. El recorrido completo de todas las pantallas se presenta en el video de demostración que acompaña este documento.

Figura 3

Flujo principal de pedido en la aplicación



Autenticación

La aplicación abre en la pantalla de inicio de sesión cuando no existe una sesión guardada. El cliente puede entrar con su correo y su contraseña, o crear una cuenta nueva desde la pantalla de registro; al registrarse, la sesión se inicia de forma automática. Las pantallas principales están protegidas, por lo que si alguien intenta entrar sin haber iniciado sesión, la aplicación lo envía a la pantalla de inicio de sesión. Al cerrar sesión, el carrito se vacía para que no pase al siguiente usuario.

Menú y selección de tamaño

El menú obtiene los productos desde la API y los agrupa por categoría. El cliente puede deslizar hacia abajo para actualizar la lista y, si no hay conexión, se muestra un mensaje con un botón para intentarlo de nuevo. Las fotografías de los productos son las mismas que se muestran en la versión web, porque ambas versiones leen el mismo catálogo.

Carrito y pago

En el carrito, el cliente puede aumentar o disminuir la cantidad de cada bebida y ver el total junto con los puntos que ganará con el pedido. Al continuar, elige el método de pago: efectivo, que se paga al recoger el pedido, o tarjeta.

El pago con tarjeta es una simulación. La aplicación muestra una tarjeta que se completa mientras el cliente escribe, valida los datos y simula el tiempo de respuesta de una pasarela de pago, pero no realiza ningún cobro. Por seguridad, el número completo de la tarjeta nunca sale del teléfono: al servidor solo se envían los últimos cuatro dígitos, para que el pedido quede registrado como pagado con tarjeta.

Confirmación y cuenta

Al confirmar el pedido, se muestra una pantalla con una animación breve, el número de pedido, el total y los puntos ganados. Se decidió que la animación fuera sutil, con un círculo que aparece con un pequeño rebote y una vibración leve, para dar seguridad al cliente sin distraerlo.

En la pestaña Cuenta, el cliente consulta sus puntos acumulados, a razón de un punto por cada dólar, y el historial de sus pedidos con su estado: preparando, listo para recoger o entregado. Mientras esta pantalla está abierta, se actualiza sola cada pocos segundos, así que cuando el administrador cambia el estado de un pedido desde la web, el cliente lo ve en su teléfono sin hacer nada.

Validaciones de formularios

Todos los formularios validan los datos antes de enviarlos. Los errores aparecen debajo de cada campo, en color rojo, y desaparecen en cuanto el usuario empieza a corregirlos. La Tabla 2 resume las reglas aplicadas.

Tabla 2

Validaciones de formularios en la aplicación

Formulario	Campo	Regla
Inicio de sesión	Correo	Obligatorio y con formato de correo válido
Inicio de sesión	Contraseña	Obligatoria
Registro	Nombre	Al menos 3 caracteres y solo letras
Registro	Correo	Formato válido y que no esté registrado
Registro	Contraseña	Al menos 6 caracteres y un número
Registro	Confirmación	Debe coincidir con la contraseña
Pago con tarjeta	Número	15 o 16 dígitos y verificación con el algoritmo de Luhn
Pago con tarjeta	Nombre	Al menos 3 caracteres
Pago con tarjeta	Vencimiento	Formato MM/AA y fecha no vencida
Pago con tarjeta	CVV	3 o 4 dígitos

Además de validar en la aplicación, la API vuelve a revisar en el servidor los datos de registro, de inicio de sesión y de los pedidos. Así, aunque alguien intentara enviar información sin pasar por la aplicación, los datos inválidos se rechazan igual.

Componentes reutilizables

Una de las prácticas más útiles fue construir la interfaz con componentes reutilizables. Por ejemplo, el componente de botón tiene una versión principal y una secundaria, y puede mostrar un indicador de carga. El componente CampoTexto muestra la etiqueta, el campo y el mensaje de error, y se usa en todos los formularios. El componente Chips sirve para elegir el tamaño del vaso y podría usarse para cualquier otra lista de opciones.

En total, la aplicación cuenta con más de quince componentes, entre ellos Pantalla, Tarjeta, Encabezado, EstadoVacio y TarjetaPedido. Gracias a ellos, cada pantalla nueva se arma combinando piezas que ya existen, y la aplicación mantiene el mismo aspecto en todas sus secciones.

Limitaciones y trabajo futuro

La principal limitación de este avance está en el almacenamiento de los datos. La API de la versión web guarda los usuarios, los pedidos y los puntos en la memoria del servidor. En el entorno local esto funciona sin problemas, pero en Vercel, donde está publicada la versión web, cada parte de la API puede ejecutarse en un proceso independiente y en varias instancias al mismo tiempo. Durante las pruebas, se observó que un pedido podía aparecer en una consulta y desaparecer en la siguiente, o que los puntos no coincidían con el historial. Además, los datos se pierden cada vez que el servidor se reinicia.

Por esta razón, las pruebas de este avance se realizaron con el servidor en ejecución local. La solución definitiva es usar una base de datos compartida, y para la siguiente entrega se integrará Firebase Authentication para el registro y el inicio de sesión, y Cloud Firestore para los productos, los pedidos y los puntos. Gracias a la arquitectura en capas, este cambio se concentrará en la carpeta de servicios y en los contextos, sin modificar las pantallas.

Como mejoras para la versión final, se considera permitir que el cliente elija la hora de recogida, agregar notas para el barista y canjear puntos por descuentos.

Conclusiones

React Native permitió llevar el proyecto al teléfono aprovechando lo que ya se conocía de React. La diferencia más grande con la web no estuvo en la lógica, sino en la forma de construir la interfaz: los componentes nativos y los estilos como objetos obligaron a pensar cada pantalla desde el teléfono, y no como una página web reducida.

Figma fue la base visual del proyecto. Aunque la estructura de la aplicación cambió respecto a los mockups, el sistema de colores, tipografías y componentes definido en la Etapa 1 se mantuvo en todas las pantallas. Esto demuestra que un buen diseño no depende de copiar cada pantalla, sino de definir reglas claras que se puedan aplicar a contextos nuevos.

Separar la aplicación en pantallas, componentes, contextos y servicios fue una de las decisiones más útiles del avance, porque permitió reutilizar el trabajo de la versión web y deja el proyecto preparado para integrar Firebase en la siguiente entrega sin rehacer el diseño. Con este avance, Café del Centro ya cuenta con una aplicación funcional para que sus clientes pidan su café, paguen y acumulen puntos desde el teléfono.

Referencias

Apple. (s. f.). *Tab bars*. Human Interface Guidelines. https://developer.apple.com/design/human-interface-guidelines/tab-bars

Expo. (s. f.). *Introduction to Expo Router*. Expo Documentation. https://docs.expo.dev/router/introduction/

Figma. (s. f.). *Figma Learn*. https://help.figma.com/

Google. (s. f.). *Navigation bar*. Material Design 3. https://m3.material.io/components/navigation-bar/overview

Hoober, S. (2013, 18 de febrero). *How do users really hold mobile devices?* UXmatters. https://www.uxmatters.com/mt/archives/2013/02/how-do-users-really-hold-mobile-devices.php

Meta Platforms. (s. f.). *Introduction*. React Native. https://reactnative.dev/docs/getting-started

React. (s. f.). *Passing data deeply with context*. https://react.dev/learn/passing-data-deeply-with-context

React Native Async Storage. (s. f.). *Usage*. https://react-native-async-storage.github.io/async-storage/docs/usage